Regular-MaestroRAG: Orchestrated Pipeline Architecture for Efficient RAG on Edge Devices

Abstract

*Retrieval-Augmented Generation (RAG) offers a compact yet powerful alternative to massive Large Language Models (LLMs), making it attractive for privacy- and resource-constrained edge deployments. However, orchestrating RAG on devices with limited compute, memory, and power is nontrivial. We address this challenge with **MaestroRAG**, a fine-grained, three-stage pipeline architecture that systematically partitions encoding, retrieval, and generation across the CPU cores and a single GPU. Our design is driven by a detailed workload characterization that reveals how to best assign each RAG stage to CPU or GPU resources, coupled with an adaptive batching strategy to balance concurrency and memory usage. We further incorporate opportunistic caching to eliminate redundant lookups and computations. Implemented on three edge platforms—including an NVIDIA Jetson and consumer-grade GPUs—MaestroRAG outperforms state-of-the-art RAG systems by up to 12× in latency, 5.6× in throughput, and 3× in energy. By efficiently leveraging CPU resources for encoding and retrieval while reserving the GPU for generation, MaestroRAG lowers resource contention and power usage, bringing on-device RAG closer to real-world viability.*

1. Introduction

Retrieval-Augmented Generation (RAG) combines compact language models with external knowledge retrieval, enabling personalized responses without embedding all knowledge in model parameters [20]. This decoupling makes RAG attractive for edge deployment, where privacy constraints favor local processing and limited resources preclude large models. In particular, users increasingly prefer on-device or edge-based AI systems because sensitive personal data can remain local, reducing exposure to centralized cloud services and improving data privacy guarantees. At the same time, service providers and corporations are also motivated to shift inference workloads toward the edge, as offloading computation to user devices reduces datacenter resource consumption, lowers operational and energy costs, and decreases token-generation overhead associated with cloud-hosted inference. However, deploying RAG efficiently on edge devices, which typically have a single GPU shared for all tasks, remains challenging; Section §2.1 defines the personal-computing edge setting we target.

Existing RAG systems, as depicted in Figure 1, place both the encoding stage (converts queries to embeddings) and the generation stage (produces responses via an LLM) on the GPU [15, 18]. On edge devices with single GPU, this creates a *structural hazard*: encoding and generation cannot run concurrently, leading to pipeline bubbles and resource underutilization. Prior systems like FlashRAG [18] and PipeRAG [15] target datacenter deployments with abundant GPUs, while EdgeRAG [34] addresses edge scenarios but neither systematically optimize CPU–GPU coordination and nor perform smart orchestration.

In this paper, we present **MaestroRAG**, a fine-grained pipeline architecture for efficient RAG inference on edge devices. Our approach is grounded in workload characterization: we profile each RAG stage and find that *encoding is compute-bound* and scales well on multicore CPUs, while *retrieval is memory-bound* with diminishing returns from parallelism. Generation, in contrast, benefits from GPU acceleration. Based on these insights, MaestroRAG assigns encoding and retrieval to dedicated CPU cores while reserving the GPU exclusively for generation. This eliminates contention and enables true pipeline parallelism.

We evaluated MaestroRAG on three platforms—NVIDIA RTX 4090, RTX 4080, and Jetson AGX Orin using realistic workloads derived from Azure traces and the Wikipedia corpus. Compared to state-of-the-art systems, MaestroRAG achieves a latency reduction of up to 12×, a performance improvement of 5.6×, and energy savings of 3×, while remaining agnostic to the choice of encoder and LLM. Our main **contributions** in this work are:

- **Workload characterization** revealing that RAG encoding is compute-bound while retrieval is memory-bound, motivating heterogeneous CPU–GPU execution.
- **A three-stage CPU–GPU pipeline** that eliminates structural hazards by partitioning encoding and retrieval across CPU cores while reserving the GPU for generation.
- **Adaptive batching and multi-worker execution** to optimize for *both* latency-critical and throughput-critical workloads on resource-constrained devices.
- **Comprehensive evaluation** on consumer-grade and embedded platforms, demonstrating significant improvements over FlashRAG [18], PipeRAG [15], and EdgeRAG [34].

2. Demystifying RAG Computation

The RAG pipeline enables LLMs to deliver enhanced personalization, use-case specificity, and customization, while preserving user data security. At its core, a compact generative model produces responses augmented with a set of `top-k` relevant knowledge items. As shown in Figure 1, the pipeline begins by encoding the user query into high-dimensional embeddings. A retriever then performs a similarity search over the user's local database to extract the `top-k` pertinent entries, which are combined with the original query and passed to the LLM to generate the final response. We next discuss the retrieval, augmentation, and generation stages in detail.

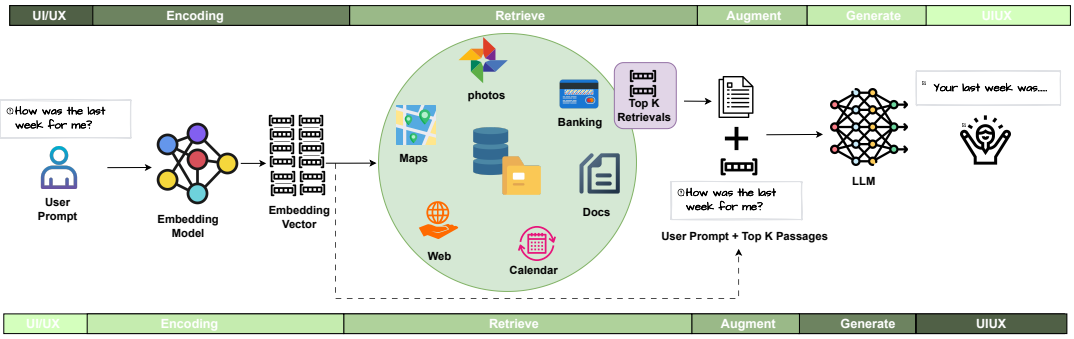


Figure 1: Classical RAG Pipeline has structural hazard and CPU under-utilization. MaestroRAG solves it with smart orchestration.

TABLE 1: Platform specifications for a local/personal-computing desktop (RTX 4090), an embedded device (Jetson AGX Orin), and a datacenter accelerator (A100). Jetson Orin's 64 GB is a *single unified* LPDDR5 pool serving CPU and GPU, not memory additional to the main-memory row.

Specification	Jetson Orin	RTX 4090	A100
CPU	Arm-A78AE	Intel i9-14900K	AMD 7742
CPU Cores	12	24	128
Main Memory (GB)	64 (unified)	128	2048
GPU VRAM (GB)		24	80
RAM Type	LPDDR5	DDR5	DDR4
VRAM Type		GDDR6X	HBM2e

2.1. Deployment Scope

We use *edge* in this paper to mean *personal-computing edge*: a local compute node running RAG for a single user, without cloud intervention, under limited power and compute budgets. The two classes of user task our design targets (Section §4) are personal-computing tasks in this sense. The desktop systems we evaluate are accordingly better described as *local/personal-computing platforms* than as embedded edge devices, and the scope extends from those platforms through embedded devices such as the Jetson AGX Orin at a 15 W cap. TABLE 1 sets both against a datacenter A100, which is not a personal compute node and is included for contrast only. Unified memory does not remove the need for the orchestration we propose: it removes the PCIe-copy cost, but neither the single-GPU encode/generate contention behind the structural hazard of Section §1, nor the competition among the encoder, the retrieval working set, and the KV cache under a constrained memory and power budget. That budget is tight: at 15 W the Orin exposes 4 of its 12 CPU cores (Section §5.1).

2.2. Different Stages of a RAG System

•**Encoding:** A RAG pipeline begins with *encoding*, where the natural-language query is tokenized into sub-word units and passed through a neural encoder (e.g., `BERT` [7], `RoBERTa` [22], `T5` [31], or compact variants like `DistilBERT` [32], `e5-base-v2` [38]), to generate high-dimensional embeddings. These embeddings are used in the retrieval stage to match against candidate knowledge elements.

•**Retrieval:** Following encoding, the system searches a *vector database* (constructed by encoding each knowledge item with the same model) to find relevant entries. This shared embedding space enables direct similarity comparisons, while database construction, being computationally intensive, is usually performed offline or during idle periods.

For efficient lookup, the system builds either hierarchical (tree-based) indices with sublinear search but higher memory cost or flat indices that perform exhaustive searches with higher time complexity. Retrieval computes a similarity metric (e.g., cosine or L2 distance [12]) between the query embedding and database entries to return `top-k` matches. Depending on resources, this computation can run on CPUs or accelerators, though CPUs often excel for large, I/O-intensive indices. Finally, the top entries are passed to the *augmentation* stage for prompt composition and generation.

•**Augmentation:** After retrieval, the augmentation stage constructs a contextualized prompt for the generation model by mapping each of the `top-k` retrieved embeddings to their original text and merging these excerpts with the user's query. Optional style or verbosity preferences (e.g., concise, detailed, formal) may also be appended. This augmented prompt enables the generation phase to produce accurate and personalized outputs using the retrieved knowledge and user instructions.

•**Generation:** The final stage of the RAG pipeline generates natural-language outputs using off-the-shelf LLMs (e.g., Llama 8B [1]). Because this stage involves parallel operations such as GEMM and self-attention, GPUs are typically used for efficient execution [6]. However, in edge devices, GPUs have limited VRAM and share resources with other tasks, so generative models are stored on device storage and loaded into GPU memory only when needed, preventing VRAM contention and improving resource utilization.

2.3. System Considerations

The fundamental RAG stages—encoding, retrieval, augmentation, and generation—are consistent across datacenter servers, local/personal-computing platforms, and embedded devices, but their execution varies widely with hardware constraints.

A typical pipeline loads the encoder model from storage into CPU memory, a significant overhead for large models under tight memory budgets. Servers usually keep both encoder and generative models resident in memory to support large batches [15, 16, 18], whereas consumer grade devices may rely on swapping, smaller models or aggressive offloading [9, 34]. Once loaded, encoding often uses GPUs for parallelism, though multicore CPUs can suffice for smaller models or workloads. CPU–GPU data transfers should be carefully managed to balance throughput against transfer costs. Retrieval, which loads an index in CPU memory to locate the `top-k` matches, is typically I/O- and memory-bound, with costs rising as the database grows or updates frequently. Augmentation, in contrast, is lightweight and merges retrieved items with the user's query and style preferences before sending the prompt to the LLM (usually on a GPU).

Batching is used to amortize overhead and improve throughput, but large batches can inflate memory use and cause thermal throttling [26, 36]. Concurrent execution across stages can reduce latency but risks pipeline back-pressure or idle cycles if stages are imbalanced. Overall, efficient RAG execution requires careful coordination of CPU- and GPU-bound stages, I/O- and memory-heavy operations, and hardware power limits. Section §3 explores these trade-offs in greater detail.

These system-level considerations motivate a detailed performance characterization, which we present next, to quantify the computational and memory demands of each RAG stage under practical deployment scenarios.

3. Detailed Characterization

This section characterizes the performance of a RAG pipeline on a desktop-class machine, highlighting how resource allocation, database size, and batching affect each stage. Our experiments use an Intel i9-14900K processor (24 physical cores), 128 GB RAM, and an RTX 4090 GPU [13, 29], with the generation length fixed at 120 tokens. To analyze performance bottlenecks, we decompose the pipeline into four stages—*encode*, *retrieve*, *augment*, and *generate*—each with distinct computational and memory demands (further detailed in Section §4). The first three stages run on the CPU, while generation executes on the GPU. Figure 2a shows the stage-wise execution time for a batch of 8 queries accessing a 4M-entry database, where each knowledge element consists of approximately 100 words. For this baseline, encoding, retrieval, and augmentation are run on a single CPU core to compare their compute cost.

3.1. Encoding Stage

The encoding stage converts user inputs into high-dimensional embeddings, with latency influenced by both batch size (`BS`) and available compute resources. Figure 2b shows that increasing `BS` over a fixed number of CPU cores (8 P-cores) leads to higher latency due to increased workload. While scaling the number of cores reduces latency, this benefit quickly saturates. Figure 2c shows diminishing returns beyond 8 cores. These results show that allocating extra cores help only for sufficiently parallel workloads.

3.2. Retrieval Stage

The retrieval stage performs a similarity search over a vector database to identify the `top-k` relevant entries. As a predominantly I/O- and memory-bound phase, it demands detailed characterization to expose performance bottlenecks. Retrieval comprises two components: (i) index fetch, which occurs once per batch and is independent of batch size, and (ii) similarity search, which runs per query. Figure 2d shows that index fetch dominates latency at small batch sizes, while similarity search grows with `BS` and becomes dominant at larger batches. At `BS=32`, total latency increases sharply due to DRAM bandwidth saturation. Figure 2e shows sub-linear latency reductions as CPU cores are increased, suggesting diminishing returns from parallelism. By contrast, Figure 2f reveals near-linear latency growth with database size, indicating stronger sensitivity to memory and I/O bandwidth than to core count. Overall, retrieval latency is governed more by data movement and cache behavior than raw compute.

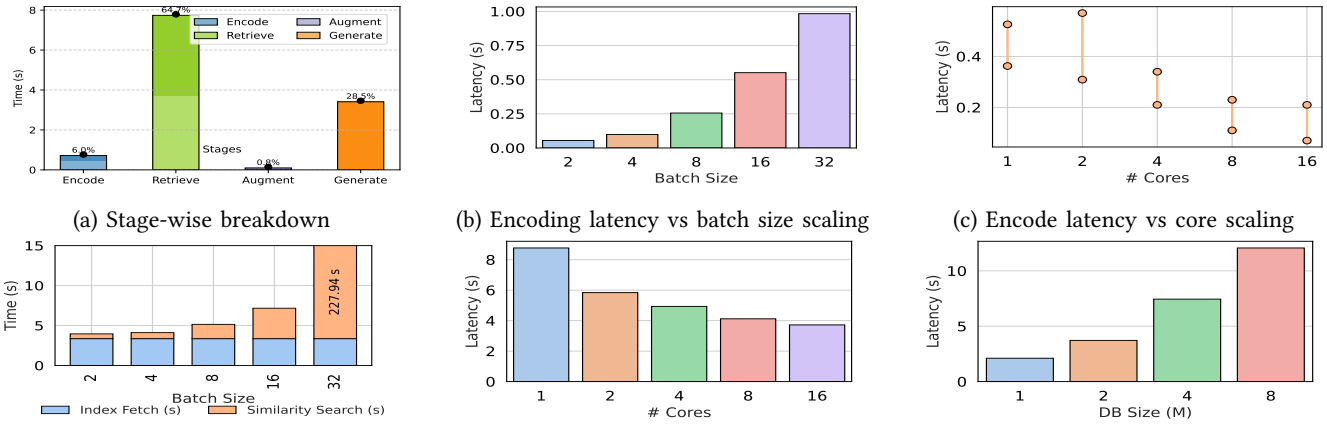
Execution Breakdown Across Stages: Figure 2e shows that adding CPU cores shortens retrieval latency up to 8 cores, after which benefits plateau because shared-resource pressure, i.e. memory bandwidth and LLC capacity, dominates. Figure 2d further decomposes retrieval into *index fetch* (I/O-bound, once per batch) and *similarity search* (compute-bound, per query). At small batches, fixed I/O cost dictates latency; at larger batches, similarity search and DRAM contention take over, revealing that retrieval performance is governed primarily by the memory hierarchy, with compute parallelism offering only secondary gains.

Impact of Batch Size and Multicore Scaling: Figure 2b illustrates that encoding enjoys linear core-utilization only when the batch size (`BS`) is large enough to saturate all cores. However, pushing `BS` beyond 16 inflates the working-set footprint, increasing cache evictions and risking DRAM thrashing; in retrieval the same threshold (Figure 2d) triggers bandwidth saturation. Thus, while larger batches amortize per-query overheads and improve throughput, they must be balanced against memory-hierarchy limits to avoid super-linear latency growth.

Effects of Database Size: Figure 2f shows nearly linear latency growth as the vector store expands from 1 M to 8 M entries on 16 cores. Once the index outgrows the cache capacity ($\approx 30MB$) and approaches DRAM, miss penalties and page faults dominate, diminishing the marginal benefit of additional cores. Hence, scaling the database, without architectural support such as tiered caching or sharding, yields a proportional increase in retrieval time, confirming that I/O, not compute, is the principal bottleneck for large knowledge stores.

3.3. Key Observations and Motivation for Design

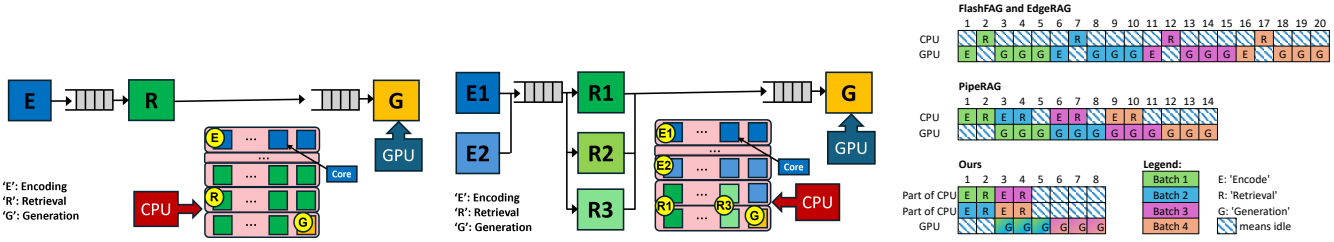
Encoding remains mostly compute-bound and scales with moderate core counts, whereas retrieval is tightly



(a) Stage-wise breakdown (b) Encoding latency vs batch size scaling (c) Encode latency vs core scaling

(d) Batch Size vs Latency in RTX 4090 (e) # Cores vs Latency in RTX-4090 (f) DB Size vs Latency in RTX 4090

Figure 2: Characterizations: (a) Stage-wise breakdown of RAG pipeline. Lighter shades represent start-up cost and darker execution time. Characterization for **encode stage** (b–c): (b) Encode stage latency for different batch sizes on 8 cores. (c) Encode latency at batch size 16 as we vary the number of CPU cores. Characterization for **retrieval stage** (d–f): (d) Retrieval stage breakdown on 4 cores with different batch sizes (*Index Fetch* vs. *Similarity Search*). (e) Retrieval latency as the number of CPU cores increases (DB size = 2 M, batch size = 8). (f) Retrieval latency vs. database size (batch size = 8).



(a) Latency-critical pipeline (b) Throughput-critical pipeline (c) MaestroRAG pipeline vs others

Figure 3: MaestroRAG pipeline: (a) latency-critical pipeline has a minimum number of parallel workers in each stage to maximize the number of CPU cores per stage. (b) throughput-critical pipeline has more number of parallel workers in the encode and retrieve stages to maximize the number of batches being processed. (c) Pipeline representation comparing prior works with MaestroRAG, where multiple batches are executed on the same hardware platform (both CPU and GPU).

coupled to memory bandwidth and database footprint, exhibiting diminishing returns from both batching and additional cores. Augmentation is lightweight and benefits from being fused with retrieval to avoid extra copies. Generation is GPU-bound but stable unless models are frequently reloaded or prompts become excessively long. Therefore, effective pipeline orchestration requires: (i) judicious CPU-core allocation between encoding and retrieval, (ii) batch sizing that maximizes utilization without breaching cache/DRAM limits, and (iii) selective offloading, only stages whose compute time exceeds transfer overheads, should migrate to the GPU. These insights inform the multi-stage scheduling strategy introduced in Section §4.

Portability of these trends to embedded platforms: The characterization above was performed on the local/personal-computing platform of Section §2.1. Because the optimizations we derive from it target CPU-side encoding and retrieval, the same trends hold on embedded platforms such as the Jetson AGX Orin. The primary difference there is the elimination of PCIe transfer overhead in unified memory systems. A similar trend is observed across SKUs and vendors.

4. Proposed Pipelined Computation Model

Here, we present the design choices behind a resource-aware framework for orchestrating the stages of RAG.

4.1. Pipeline Design Overview

Our pipeline comprises three stages: *Encode*, *Retrieve+Augment*, and *Generate*. The rationality of our three-stage design is based on our characterization study, which reveals that the *Encode* stage typically involves computationally intensive floating-point and matrix operations, while the *Retrieve* process is largely memory-bound, requiring searches through lists of indices. These two operations, if implemented together on the same CPU cores, can interfere with each other's cache contents, causing higher miss latencies. By dedicating an *exclusive* set of CPU cores to each stage (by specifying the core affinity to the process running), we exploit the reuse of model parameters and index data

within those cores across batches without polluting caches and by containing the working set of that process within the private cache hierarchy of those cores. Consequently, we separate these stages from the conventional single *retrieve* step often seen in RAG pipelines.

Since the *Augment* operation (2.2) shares a working set similar to retrieval, we opt to merge *Retrieve* and *Augment* into a single stage. Although running retrieval and augmentation as distinct processes can, in theory, offer benefits in terms of cache isolation, the overhead of inter-process communication and synchronization often outweighs the gains. Therefore, *Encode* and *Retrieve+Augment* each run on a dedicated set of CPU cores, followed by the *Generate* stage, ensuring efficient use of system resources at every step. Separating out augment and retrieval stages, although brings cache hit benefits in their respective private caches, but suffers from synchronization and data transfer overheads across processes (resulting in latency increase of the two stages – up to 128 ms on average), overshadowing the benefits from cache hits.

Our design for an edge personal computing device targets two main types of user tasks: latency-critical and throughput-critical. A latency-critical task involves a virtual assistant that provides real-time insights, such as calendar summaries. A throughput-critical task focuses on generating recommendations for shopping, entertainment, or financial investments based on data from e-commerce, entertainment, and banking apps. The insights developed in the previous characterization study help us to use the multicore scalability of stages to our advantage to move towards a more balanced pipeline organization. As shown in Figure 3a and Figure 3b, we introduce a resource-aware multi-width pipeline, where each stage can be executed in parallel on different numbers of CPU cores. Here, each 'worker' is a process running independently on allocated cores capable of processing one batch at a time, and there can be a variable number of workers in each pipeline stage depending on the number of cores the CPU has and the type of latency/throughput need of the application. We develop different schemes to tackle the latency and throughput critical situations. For latency-critical workloads, we allocate maximum CPU cores to each worker in a single-width pipeline to minimize response time. For throughput-critical workloads, diverging from traditional pipeline design, we spawn multiple workers per stage with reduced per-worker core allocation, as shown in Figure 3b. This configuration enables multiple request batches to be processed concurrently within the pipeline, compared to single-batch processing in the latency-optimized case (§4.2).

Our pipeline design differs from state-of-the-art methods as shown in Figure 3c. Existing pipelines typically perform encoding and retrieval on either CPUs or GPUs, followed by CPU-based augmentation and GPU-based generation [15, 18, 34]. FlashRAG and EdgeRAG utilize GPUs for encoding and generation, while performing retrieval on CPUs. PipeRAG, with certain optimizations, can disaggregate encoding and retrieval stages on CPUs followed by GPU-based generation, reducing pipeline bubbles compared to prior solutions. However, latency differences between stages and the synchronous pipeline nature do not fully eliminate bubbles. Our approach addresses this by orchestrating an asynchronous pipeline, where each stage receives compute resources proportional to its computational requirements.

Adaptive batching: The computationally bound encoding stage strongly correlates its latency with the amount of core provisioning and batch size. In contrast, that of the retrieval stage depends more heavily on the size of I/O and memory operations; as shown in Figure 2e, doubling the core count from 8 to 16 generates diminishing benefits for latency. Thus, a one-size-fits-all approach to batch size and core allocation is not optimal across different databases, batch profiles, models, and hardware architectures. Instead, a more flexible and judicious compute allocation strategy is required, taking into account the specific operation, service-level objective (SLO) constraints, hardware details, and the RAG model itself. Leveraging the profiling results discussed in Section §3, we identify the "sweet spot" for batch size and core allocation for each stage.

However, some requests exceed the available provisioning for their batch size or SLO constraints. In such cases, we split large batches into smaller quanta, maximizing resource usage and minimizing overall latency. These quanta are processed asynchronously to avoid lock-step overhead: whenever a worker finds tasks in its input queue, it proceeds immediately, regardless of any straggling peers. This is especially beneficial for large batches, where the retrieval stage would otherwise suffer significant I/O overhead. By breaking large I/O requests into smaller chunks, individual retrievals complete faster, and the staggered read requests reduce the risk of saturating memory bandwidth.

Since the memory capacity of the GPU in a typical personal computing platform is smaller, it is capable of generating responses for a limited number of prompts before it goes out of memory. Adaptive batching alleviates this bottleneck by breaking down the larger batch of requests for a GPU into smaller quanta and scheduling them. For the GPU bound jobs, typically the process running on a CPU acts as an agent to only launch kernels and collect the data back. Given their lightweight, bursty nature, these tasks are best scheduled on E-cores, which are typically reserved for non-intensive workloads. Generation latency depends on the intended prompt length. The mapper profiles T_G across those lengths (Section §4.2), and adaptive batching creates memory-safe GPU quanta. Worker allocation is nonetheless static within a session, following the start-up amortization rationale of Section §4.3. Remapping cores at run time under context drift is a future extension.

4.2. Resource Mapping

Our design assigns the CPU–based stages,*encoding* (*E*) and *retrieval+augmentation* (*RA*), to CPU cores, while the *generation* (*G*) stage runs on GPU. Modern CPUs often feature *performance cores (P-cores)* for high throughput and *efficiency cores (E-cores)* for low power usage. We denote their counts by N_P and N_E, respectively, so that $N = N_P + N_E$ is the total number of CPU cores. We

profile the latency $T_s(x, y)$ of each CPU–bound stage $s \in \{E, RA\}$ under different allocations of x P-cores and y E-cores. For generation, we measure T_G given the intended prompt lengths or batch sizes. These profiles are re-measured whenever the hardware configuration or LLM changes, keeping the system *model-aware*. For latency-sensitive batches, the end-to-end time is given by $T_{\text{E2E}}(x_E, y_E,\ x_{RA}, y_{RA}) = T_E(x_E, y_E) + T_{RA}(x_{RA}, y_{RA}) + T_G$, subject to $x_E + x_{RA} \leq N_P$ and $y_E + y_{RA} \leq N_E$. One can choose (x_s, y_s) per stage $s \in \{E, RA\}$ to minimize T_{E2E} or meet a latency SLO $L_{\max}$, typically by using a small grid search. For sustained workloads, each CPU–bound stage $s \in \{E, RA\}$ may run w_s workers, each allocated $(\tilde{x}_s, \tilde{y}_s)$ P- and E-cores. The throughput is: $\text{TH}_s = \frac{w_s}{T_s(\tilde{x}_s, \tilde{y}_s)}$, subject to $\sum_s w_s \tilde{x}_s \leq N_P$ and $\sum_s w_s \tilde{y}_s \leq N_E$. We then maximize the minimum throughput across all stages for balanced performance. In our experiments, we treat all CPU cores uniformly (i.e., $y = 0$), but this design naturally extends to systems where P- and E-cores differ in performance.

As an example, applying the above equations to our desktop-grade CPU platform, we determine the optimal core allocation strategy for the latency-critical application to be **E:8, RA:22, G:1**[1], with the remaining single core dedicated to the scheduler. Under this mapper-identified configuration, our proposed three-stage pipeline (`for DB=2M; BS=8`) results in an execution time of 1.178 s with a stage-wise breakdown of ⟨E=0.21s, RA=0.78s⟩ and a scheduler overhead of 0.188s. In comparison, the standard four-stage pipeline requires 1.448 s with the breakdown ⟨E=0.192s, R=0.761s, A=0.104s⟩ along with scheduler overhead of 0.391s. Both configurations ignore the **G** stage, as its execution time remains constant. Notably, our three-stage pipeline delivers an approximate 22% improvement over the four-stage approach, primarily due to the reduced scheduler overhead. This performance is also significantly better than the naïve allocation strategy, which requires 9.5 s (8.06×) for similar CPU execution.

Multi-Worker Duplication: Although dedicating more cores can reduce latency for one worker, we often scale *width* by creating additional workers on fewer cores, thereby feeding the pipeline more batches and avoiding CPU underutilization. In practice, the generation stage is often slowest; however, limiting encode/retrieve to modest core counts often yields better overall throughput, since more workers can run in parallel. Duplication can inflate memory use if each worker loads its own model or index; therefore, optimizations (in §4.3) to share these large data structures between processes. For instance, as depicted in Figure 3a, allocating 8 cores per worker for stage E, 22 cores per worker for stage RA, and 1 core per worker for stage G achieves a latency of 4.898s. In contrast, the configuration in Figure 3b with 4 cores per worker for stage E (2 workers), 6 cores per worker for stage RA (3 workers), and 1 core per worker for stage G (1 worker)—yields a latency of 5.347s while serving 3× the number of requests on Intel i9-14900K platform paired with RTX 4090 GPU.

Adaptive Mapping: Once $T_s(x, y)$ and T_G have been profiled, we solve either a latency-oriented or a throughput-oriented formulation under the same CPU constraints. We then *pin* each stage (or worker group) to its allocated P- and E-cores. Re-running this procedure on a new device or LLM keeps the pipeline *hardware-aware* as well as *model-aware,* consistently aligned with SLO targets. The benefits of the proposed mapping are further discussed in Section §5.9.

4.3. System and Software Optimizations

Our pipeline orchestration lies in the allocation of the available CPU cores and assigning them to workers (processes running the stage) judiciously. For applications that are throughput critical, we make redundant workers to increase the width of the software pipeline. On the other hand, for latency-critical applications, we allocate maximal CPU cores to reap the most benefit in latency. We observe a few inefficiencies in the naive orchestration of this pipeline. First, the worker initialization process that comprises configuring the threads, loading the encoding model, and fetching the indices from the disk to the main memory is expensive. Having multiple such workers leads to redundant copies of the same data. Second, breaking down the requests of larger sizes into smaller quanta leads to creation of multiple batches, which in essence lead to multiple start-up costs that can impact the throughput of the system. In order to address these inefficiencies, we implement several orchestration optimizations:

Shared data access: Multiple workers in encoding and retrieval stages exhibit a high reuse of the same model and indices, respectively. A naive way of execution would load these data privately to the respective processes and access them. We implemented model weight sharing across processes through a shared memory mechanism, eliminating redundant copies of large model parameters, which saves us the time to fetch the same.

Memory mapped indices: This approach maps the index file directly to the virtual memory space without completely loading it into RAM. The operating system handles paging as needed, bringing only the accessed portions to physical memory. This enables our system to work with indices that are significantly larger than available RAM while maintaining performance through OS-level caching mechanisms.

Start-up costs: Launching workers per arrived batch leads to multiple start-up costs compared to the actual processing time. Since our classification (latency and throughput critical) remains static throughout the request, the number of workers can be fixed apriori and need not change during the execution of the successive batches. Separating out the worker initialization from the task execution causes these costly start-up operations to happen only once before any requests are even launched. It might impact the execution time if the nature of the application changes on the fly. But this is a design decision that we expect the user to make that presents a reasonable tradeoff in most real-world

1. With Jetson at a 15W power budget (see Section §5.1), we get 4 cores to work with of which 2 are used for **E**, 1 for **RA** and 1 for **G** and scheduler.

scenarios, where application requirements tend to remain consistent throughout an operational session.

5. Implementation and Evaluation

5.1. Implementation Details

We implement MaestroRAG in Python (about 6,000 lines of code), mainly using PyTorch for ML operations and FAISS for high-speed vector similarity search. A key optimization is to place the model weights and intermediate tensors in a shared memory region, so that multiple processes in the same pipeline stage can load them without duplicating. This reduces overall memory usage by ∼70% compared to loading models into each process privately, and with minimal synchronization overhead. To manage parallel execution across pipeline stages, we avoid Python's `ThreadPoolExecutor` and use a `ProcessPoolExecutor` configuration along with explicit CPU-core pinning. This approach circumvents the Global Interpreter Lock, ensures genuine parallelism on each CPU core, and reduces cache misses by lowering context-switch overhead. In the encoding stage, we store the embeddings in contiguous arrays via `np.ascontiguousarray()`, which helps to exploit SIMD-based vector instructions more effectively.

Deployment System: Following Section §2.1, our evaluations span two local/personal-computing platforms and one embedded device: (1) An Intel i9-14900K platform [13] with 32 virtual cores (8 P-cores and 16 E-cores) and 128 GB of main memory, paired with an RTX 4090 GPU [29] with 24 GB of VRAM; (2) An Intel i9-14900F with the same CPU-core configuration but connected to an RTX 4080 GPU with 16 GB of VRAM; and (3) An Nvidia Jetson AGX Orin development kit with 64 GB of unified memory [28], where we cap its power budget at the lowest provisioned state of 15W. All three are personal-computing edge nodes in the sense of Section §2.1: each runs RAG locally for a single user, without cloud intervention, under limited power and compute budgets. On the Jetson the budget is explicit: the 15W cap leaves 4 of its 12 CPU cores available.

Dataset and Traces: We used the Wikipedia corpus dump [39] as our knowledge source and relied on the FAISS library [8] to build a *flat index* that uses an L2-distance measure for `top-k` similarity searches. Query prompts are drawn from Google's Natural Question Dataset [19]. Because there is no publicly available trace for RAG workloads on personal devices, we downscaled bursty traces from Azure to better approximate smaller, consumer-grade computing capabilities. For the similarity search process, we use flat indices to obtain the `top-k` most similar elements. The text embedding model used in all experiments is `e5-base-v2` [38]. For the generation stage, we used Llama-3.1-8B-Instruct [1] and OPT 2.7B [41] on the RTX 4090 and RTX 4080, respectively, and a smaller Llama-3.2-1B [24] in Jetson Orin, where we impose a 15W power limit.

Metrics: We report two key performance metrics for the end-to-end pipeline. The first is "latency", which is the total completion time for a batch of queries. The second is "throughput" (qps), defined as the number of queries processed per second. In the results, we compare our proposed MaestroRAG against three baseline approaches: (1) FlashRAG [18]–presents a modular RAG framework where both encoding and generation are placed on the GPU; (2) PipeRAG [15]–a server-oriented pipeline design; and (3) EdgeRAG [34]–mitigates memory usage by caching only coarse indices and generating finer embeddings on demand using the GPU. To ensure that the proposed orchestration fits in the tight power and energy constraints of the personal-computing edge platforms, we report the peak and average power (in W) and the total energy (in J) consumed per inference query.

TABLE 2: Stage-level latency on the RTX 4090 at `DB=4 M` and `BS=8`, with caching disabled. Generation is excluded because its settings are common to all four systems. A spanned cell marks stages a system measures together: MaestroRAG and EdgeRAG [34] fuse retrieval and augmentation (Section §4.1), while FlashRAG [18] fuses encoding and retrieval. An empty cell marks a cost the system does not report separately.

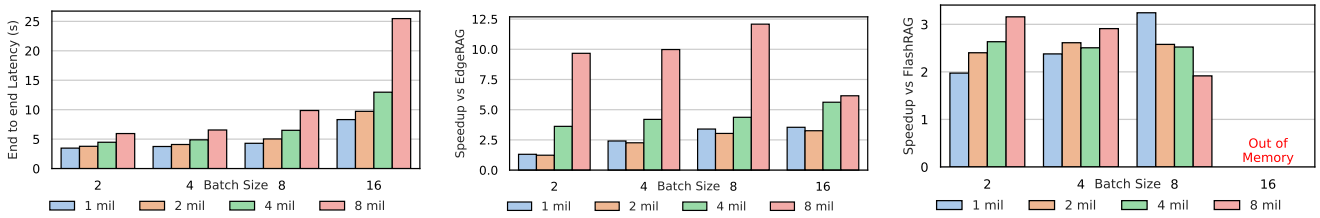
System	Encode	Retrieve	Augment	Model loads	Scheduler / sync
MaestroRAG	0.20 s	1.60 s			0.20 s
EdgeRAG [34]	0.28 s	25.19 s			
FlashRAG [18]	7.26 s		0.10 s	0.73 s (encoder) 2.20 s (LLM)	
PipeRAG [15]	6.20 s	5.20 s	0.10 s		$\leq$ 2 s

5.2. Stage-Level Latency Breakdown

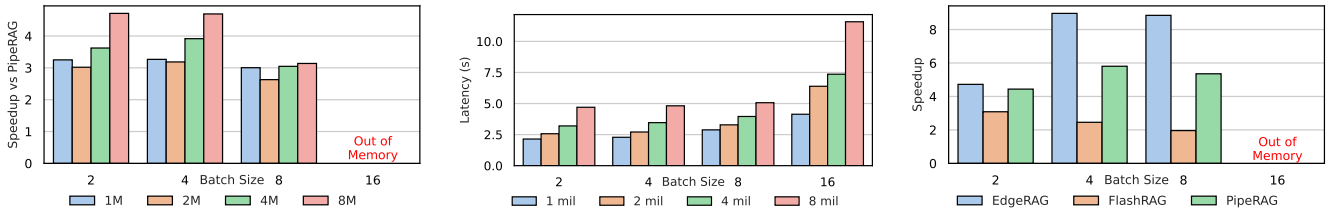
TABLE 2 decomposes the per-stage cost of MaestroRAG and the three baselines at a common operating point. Generation is excluded because its settings are the same for all four systems, so it is not a source of difference between them. Caching was disabled for these measurements and for the primary results reported in this section. EdgeRAG [34] spends 25.19 s in fused retrieval and augmentation, which is the cost of generating embeddings on demand. FlashRAG [18] pays 0.73 s to load the encoder and a further 2.20 s to load the LLM, which are model reloads. PipeRAG [15] measures the three CPU stages separately but pays up to 2 s in synchronization, serialization, and timeout, which is contention between stages. MaestroRAG reports 0.20 s for encoding, 1.60 s for fused retrieval and augmentation, and 0.20 s of scheduler cost.

5.3. Results on RTX 4090 and RTX 4080 Desktop

Overall Latency and Speedups on RTX 4090: Figure 4a plots the end-to-end latency of our design on an RTX 4090 across batch sizes of 2–16 and database sizes of 1–8 million. As batch size or database size grows, the latency increases because larger batches demand more encoding and generation while bigger databases incur heavier I/O in retrieval. Even in the largest case (`BS=16`, `DB=8 M`), our approach remains feasible, completing in $\approx 25\,s$. In Figure 4b–4d, we show speedups over EdgeRAG, FlashRAG, and PipeRAG, respectively, and observe consistent gains. Compared to EdgeRAG [Figure 4(b)], we see up to a $\approx 12\times$ speedup at



(a) E2E latency of MaestroRAG in RTX4090. (b) Speedup: MaestroRAG vs EdgeRAG [34]. (c) Speedup: MaestroRAG vs FlashRAG [18].



(d) Speedup: MaestroRAG vs PipeRAG [15]. (e) E2E latency of MaestroRAG in RTX 4080 (f) Speedup: MaestroRAG vs other baselines.

Figure 4: Speedup of MaestroRAG relative to state-of-the-art RAG frameworks on consumer-grade systems with RTX 4090 (a–d) and RTX 4080 (e–f) GPU. All speedups observed for MaestroRAG results from a concoction of optimizations presented in Section §4.1, Section §4.2, and Section §4.3). Caching was disabled during all evaluations to emulate random query patterns that lack spatio-temporal locality, representing a worst-case scenario commonly observed when extending from personal devices to large-scale serving frameworks. For MaestroRAG, we deploy one worker per pipeline stage with ⟨**8**, **22**, and **1**⟩ cores for the ⟨**E**, **RA**, and **G**⟩ stages, respectively. The **G** stage's allocated core is responsible for launching generation workers on the GPU and collecting results, while the system scheduler runs on the remaining available core.

BS=8, DB=8 M, primarily because our CPU-based encoding avoids redundant GPU transfers. In higher batches, the speedup drops slightly (e.g., to 6.15× at BS=16, DB=8 M) due to partial I/O saturation, but still substantially outperforms EdgeRAG. Against FlashRAG [Figure 4(c)], we see gains of 2–3× and occasionally higher when larger batches make GPU memory usage critical; FlashRAG often runs out of memory for BS=16, whereas our CPU-side retrieval and encoding remain robust. These experiments highlight the importance of selecting which stages run on the GPU and which on the CPU. Pushing both encoder and LLM to the GPU triggers repeated data transfers and frequent model reloading, while moderate batch sizes (BS=4–8) help balance resource use. Overall, carefully partitioning CPU vs. GPU tasks and choosing middle-range batch sizes yields 6–12× faster processing without risking out-of-memory problems; Section §5.2 decomposes these results by pipeline stage. Two major lessons emerge from Figure 4. First, partitioning CPU and GPU tasks carefully is crucial: forcing both encoder and generative model onto the GPU triggers heavy data transfers and repeated model loads. Second, moderate batch sizes (BS=4–8) typically provide the most efficient balance of I/O and compute; pushing too high eventually saturates I/O or memory, but staying too low underutilizes resources. Overall, from a *designer's perspective*, sweet-spot batching and CPU–GPU division together yield 6–12× latency gains over existing designs without risking out-of-memory failures.

Results on an RTX 4080 Desktop: Figure 4e shows that, on RTX 4080, our total latency follows a similar pattern, scaling to about 11.58 s at BS=16, DB=8 M. In Figure 4f, where we specifically compare speedups at DB=4 M, batch sizes of 2–8 run successfully with MaestroRAG but cause memory exhaustion in all baselines at BS=16. That reflects how naive GPU offloading can overwhelm the 16 GB GPU memory if encoding and retrieval also reside there. In contrast, dedicating only generation to the GPU keeps our pipeline stable for larger batches. These results confirm that CPU-driven encoding and retrieval strategies are more sustainable for mid-range GPUs while still providing significant performance gains and avoiding failures on bigger inputs.

5.4. Results on Jetson AGX Orin

Although absolute latencies on Jetson are inevitably higher (tens of seconds instead of single digits), we observe speedups of up to 1.35× at BS=8, declining to about 1.25× at BS=16 due to resource saturation in both CPU and memory. These results confirm that minimizing CPU–GPU data transfers remains advantageous in low-power SoCs and that judicious model and index selections are crucial in memory-constrained scenarios, consistent with the trend portability established in Section §3.3. By confining encoding and retrieval to the CPU and using memory-mapped indices, Figure 5 contrasts our pipeline with EdgeRAG on a Jetson AGX Orin limited to 4 CPU cores. we eliminate large data copies and cut overhead relative to EdgeRAG by 25%–35%.

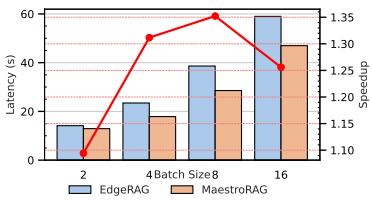


Figure 5: Latency comparison of EdgeRAG and MaestroRAG on a Jetson AGX Orin with 1 M database.

Although gains are smaller than on larger GPUs, they illustrate that our pipeline and data-sharing optimizations still yield meaningful benefits on edge devices.

5.5. Main Latency Results

Figure 7b reports end-to-end latencies for MaestroRAG and three baselines on (i) an RTX 4090 with a 4 M database at BS=8, (ii) an RTX 4080 also at BS=8 and 4 M, and (iii) Jetson AGX Orin with 1 M at BS=8. On the RTX 4090, our method completes inference in 6.50 s, which is 3–4× faster than FlashRAG [18] (16.39 s) or PipeRAG [15] (19.80 s), and $\geq$ 4× faster than EdgeRAG [34], whose stage-level composition is given in Section §5.2. On the RTX 4080, our pipeline finishes in 3.96 s, outperforming EdgeRAG [34], FlashRAG [18], and PipeRAG [15] by up to 8.8×, 1.9×, and 5.4×, respectively. On Jetson AGX Orin at BS=8 and 1 M, MaestroRAG lowers latency to 28.56 s, a 1.35× improvement over EdgeRAG [34] (38.62 s), even under a tight 15 W power cap and limited CPU/GPU capacity. We also measure tail latency and amplification (*P99*, *P99/P50*) at BS=8, DB=2 M. Our system achieves 5.26 s (1.04), outperforming FlashRAG (14.86 s, 1.14) and PipeRAG (14.41 s, 1.09). These findings confirm that our pipeline achieves robust, memory-aware performance across heterogeneous platforms.

5.6. Throughput Evaluation

To assess throughput, we simulate bursty traffic from scaled Azure traces [25], processing 256 queries on the RTX 4090 and 128 in Jetson, injected every 10 s and 15 s, respectively. As shown in Figure 7c, we set DB=4 M and BS=8, consistent with the mid-scale latency experiments. On the RTX 4090, MaestroRAG reaches 1.60 QPS, clearly ahead of EdgeRAG [34] at 0.29 QPS, FlashRAG [18] at 0.68 QPS, and PipeRAG [15] at 1.19 QPS. Although the latter two are more GPU-conscious, they still incur frequent data movements and memory overheads that collectively restrict concurrency under large-batch bursts. In contrast, our CPU-based encoding and retrieval allow asynchronous, multi-query execution without overburdening GPU memory. In Jetson, MaestroRAG achieves 0.43 QPS, which is 6.7× better than EdgeRAG [34] (0.064 QPS) and about 1.16× faster than PipeRAG [15] (0.37 QPS). FlashRAG [18] relies on vLLM, which is not supported on Jetson; so, it is omitted. Despite the overall lower speed on the Jetson, our approach clearly shows that decoupling retrieval and encoding on the CPU increases concurrency under limited DRAM, thus sustaining higher steady-state throughput. These gains highlight the importance of a fine-grained pipeline with minimal CPU–GPU transfers, which helps the system respond more gracefully to bursty arrivals.

5.7. Power/energy efficiency

To assess the real-world feasibility of MaestroRAG in terms of energy consumption on edge platforms, we perform a complete power/energy profile of both the CPU and the GPU. For CPU power measurement, we employ `turbostat` utility [37], which provides hardware-based power consumption data by accessing Model-Specific Registers and Running Average Power Limit counters. This approach delivers package- and core-level power measurements with sampling at a 10Hz frequency. GPU power consumption is measured using the `nvidia-smi`, which queries on-board power sensors to obtain instantaneous power draw at *20Hz* sampling frequency. For the Jetson platform, we cap the power to 15*W* only, which is the lowest power setting offered on the particular platform representative of the handheld mobile devices used everyday.

The power and energy measurement results are plotted in Figure 6. MaestroRAG significantly outperforms FlashRAG and PipeRAG in energy efficiency, only consuming 253.3*J*/*query*, compared to 792.00*J* and 780.92*J*/*query* for FlashRAG and PipeRAG, respectively. Moreover, FlashRAG and PipeRAG exhibit 16.36% and 8.08% higher peak CPU power usage, respectively. Attributing CPU package energy to pipeline stages, after subtracting idle package power and with DRAM excluded, gives 79.2% retrieval and augmentation, 19.1% generation-driving, and 1.8% other for FlashRAG; 20.4% encoding, 64.5% retrieval, 0% augmentation, and 15.6% generation for PipeRAG; and 5.5% encoding, 83.2% fused retrieval and augmentation, and 11.2% generation for MaestroRAG. These shares are of idle-subtracted energy, so they are reported as proportions rather than as a decomposition of the absolute totals above. The power consumption results for EdgeRAG are excluded from our evaluation due to implementation differences that preclude a fair comparison. EdgeRAG employs an on-demand embedding generation strategy that computes target cluster embeddings during cache misses, introducing additional computational overhead that substantially increases the system's power profile compared to MaestroRAG. Because that path performs additional, non-equivalent work, a per-stage energy attribution for EdgeRAG [34] would not be comparable with the other three systems.

TABLE 3: Latency with caching at BS=1, with a TTL of 300 s, a cache capacity of 32 entries, and 5 retrieved documents per prompt. MaestroRAG: MR; EdgeRAG: ER

	2 mil		4 mil		8 mil	
Method	**ER**	**MR**	**ER**	**MR**	**ER**	**MR**
Exact	2.033s	0.87s	2.0326s	0.92s	2.0335s	0.887s
Sim	2.047s	3.116s	2.003s	3.061s	2.138s	3.089s

5.8. Software Caching

To further reduce redundant computations and memory transfers, we exploit software caching in our design. By storing intermediate results in a cache, we avoid repeated operations such as costly vector similarity searches, thereby improving latency and throughput. Since edge deployments often process successive queries with overlapping context, caching naturally complements our design choices. This sec-

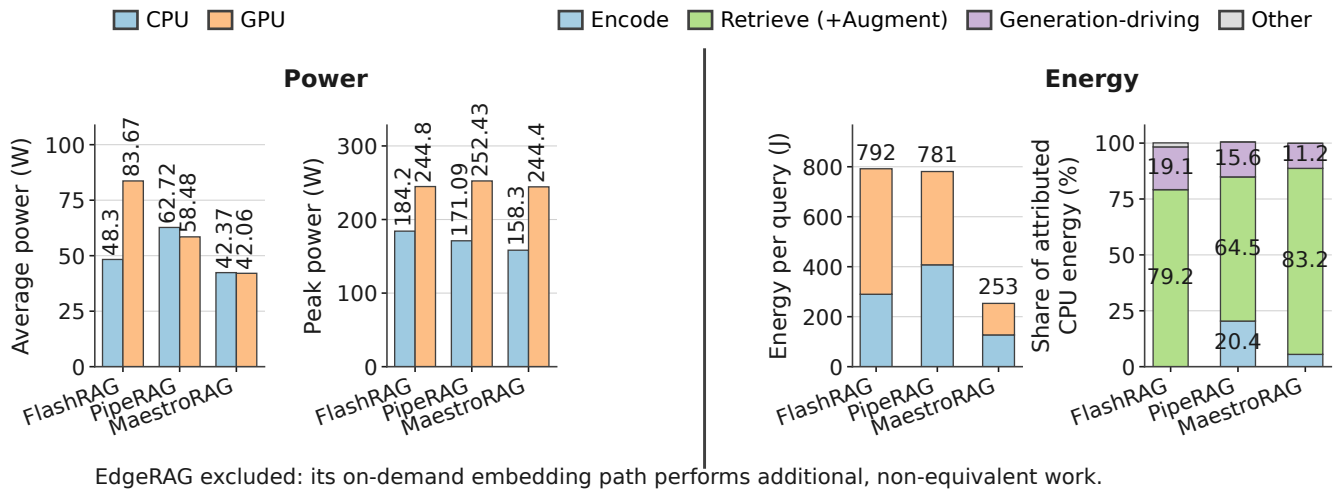
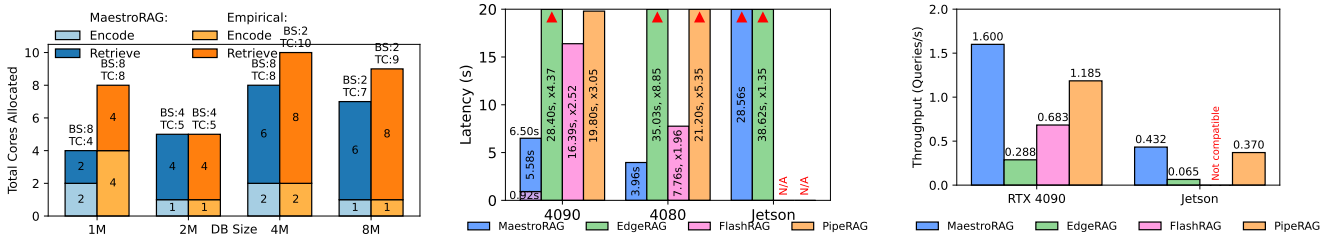


Figure 6: Power and energy on the Intel i9-14900K platform with an RTX 4090 GPU at BS=2, DB=4 M, and top-k=5. Left of the rule, average and peak power for the CPU and the GPU. Right of the rule, energy per query and the composition of CPU energy by pipeline stage. Power and energy are measured at the CPU package and at the GPU board. The composition is a share of idle-subtracted CPU package energy with DRAM excluded, so it is drawn on its own normalized axis rather than stacked onto the absolute joules beside it. EdgeRAG [34] is excluded throughout.



(a) Adaptive Worker Allocation (b) End to end latency comparison (c) Throughput comparison

Figure 7: Key findings: (a) The adaptive worker allocation technique often results in better resource partitioning combinations to achieve the same latency/throughput. (b) End-to-end latency comparison for configurations selected by the adaptive worker allocator. (c) Throughput comparison for configurations selected by the adaptive worker allocator.

tion outlines our caching strategies, adapted from previous work on cache-augmented generation [4].

Model caching, where the encoding model is loaded once into a shared memory region, accessible to all workers, eliminates redundant memory copies and re-initialization overhead, accelerating query handling by avoiding frequent model load/unload cycles. RAG workloads on edge might exhibit predictable patterns in query type, frequency, and likelihood (e.g. asking about the weather multiple times) [34]. Although in practice this is highly application dependent, reusing previously fetched or generated data could significantly boost both performance and energy savings. We adopt two complementary data caching schemes.

Similarity/Partial match retrieval caching: For semantically similar queries, the system reuses prior retrieval results if the new query's embedding exceeds a cosine similarity threshold (e.g., 90%) with a cached embedding. Each cache entry stores the query embedding, retrieved documents, and a user-defined time-to-live (TTL). Thresholds and TTLs are tunable to balance factual accuracy and latency, as supported by [3, 33]. This scheme bypasses the retrieval phase, the most latency-intensive step, while still performing a fresh generation for nuanced differences.

Exact/Complete match caching: If a query matches a cached one exactly (embedding-based), the pipeline returns the stored final output directly, skipping both retrieval and generation. This assumes the vector database has not changed since the previous query, which is likely within the TTL window of a few minutes. This method is especially effective for repeated prompts, completely avoiding decoding overhead.

TABLE 3 reports the results of our caching mechanisms under exact- and similarity-match scenarios. All experiments use TTL of 300 s and cache capacity of 32 entries, each storing 5 retrieved documents per prompt. For a fair comparison with EdgeRAG [34], we adopt a batch size of 1. Exact matching returns the cached final answer and skips both retrieval and generation, placing MaestroRAG between 0.87 s and 0.92 s. Similarity matching reuses only the top-k retrieved documents and generates afresh for the new query, placing it between 3.06 s and 3.12 s. The difference between the two is therefore the cost of generation, which similarity

matching still incurs; it is not a RAM-capacity effect. Against EdgeRAG [34], the residual gap in the similarity-match case is our process and thread handoff across workers, which adds approximately 1.1 s. EdgeRAG [34] does not incur this cost, since its execution is largely sequential and involves no process orchestration or thread synchronization. Because the measurement uses a batch size of 1, it compares our worst case against EdgeRAG [34]'s nominal case; the handoff is a fixed per-batch cost, so we expect it to amortize at larger batch sizes. When 80% of queries exhibit strong similarity, average latency improves substantially despite occasional cache misses. On the Jetson AGX Orin (15 W power cap), caching provides consistent speedups by avoiding redundant encoder invocations and memory transfers. Our cache management exploits temporal and semantic locality. Overall, the cache policy provides a tunable, memory-efficient approach that leverages session-level query locality, yielding significant performance gains for interactive edge deployments. Figure 7b (purple stacked bar) showcases a pathological best-case exact match example on the RTX 4090, where exact-match caching reduces latency from 6.50 s to under 1 s for repeated or highly similar queries.

5.9. Additional Insights

Here, we provide additional results that highlight the adaptability and generalization capacity of our design across a range of sensitivity settings. These include experiments with more relaxed indexing strategies and more stringent configurations involving longer input context lengths.

Latency results insights: MaestroRAG yields latency reductions of 1.25×–12× across three hardware platforms. Avoiding naive GPU offloading is crucial: we place only the generation stage on the GPU to limit data transfers and loading overheads, while CPU-driven retrieval and encoding prevent out-of-memory failures, even at BS=16 with large databases. Furthermore, moderate batch sizes (BS=4–8) strike the best performance balance, as oversized batches saturate I/O and small ones squander concurrency. Although Jetson-class hardware is relatively constrained and possesses unified memory, where both encoder and generation models are stored, applying the proposed software optimizations and distributing appropriate compute over CPU and GPU resources brings a speedup of 1.35× over EdgeRAG.

Adaptive Batching and Worker Allocation: In Figure 7a, we compare the recommended batch-size and core-allocation between our adaptive mapper (§4.2) and exhaustive grid searching. The bars labeled "Mapper Identified" show the mapper's suggested allocations, while the "Empirically Observed" bars come from direct measurement. The results on the RTX 4090 for four database sizes (1 M, 2 M, 4 M, 8 M) indicate that the mapper usually matches or outperforms purely empirical approaches, particularly at 2 M and 4 M. For instance, with a 4 M database, the mapper selects BS=8 and assigns 2 cores to encoding and 6 cores to retrieval, which indeed yields strong measured performance. Occasionally the mapper opts for fewer encoding cores to keep the retrieval scaling robust. Exhaustive grid searches quickly become infeasible on real systems, especially as the database or query volume increases. In contrast, the mapper adapts well to new hardware or changing data sizes, making it especially useful in constrained environments that cannot afford extensive trial configurations.

Impact of number of tokens: To assess the scalability of our approach with respect to input length, we vary the number of input tokens while keeping all other hyperparameters fixed. The encoding latency increases only 0.6% when moving from 10 to 20 tokens, and 1.2% at 100 tokens. Generation latency remains unchanged at 20 tokens and increases by 12% at 100 tokens. Even with 100 tokens, the total input to the generation module (query and the top-5 retrieved passages) exceeds 500 tokens, indicating the system's resilience to context length expansion.

Impact of indexing schemes: In addition to the flat indexing scheme for accessing the database, we also experimented with three other indexing schemes—approximate nearest neighbor (ANN) methods [8], IVF-Flat, IVF-PQ, and HNSW—to understand the performance implications. All alternatives provided substantial latency improvements, with observed gains of 29.27%, 28.15%, and 32.18% respectively. This demonstrates the ability of our system to trade off between retrieval accuracy and latency via configurable indexing.

Impact of optimization mechanisms: To isolate the impact of individual optimizations, we performed an ablation study of end-to-end latency. Starting from a baseline latency of 15.12s (no optimizations), we observe incremental improvements: adaptive batching and core allocation reduce the latency to 11.79s; software-level optimizations bring it down further to 6.497s; and (effective) caching achieves 2.003s. These results clearly demonstrate the effectiveness of our layered optimizations.

Portability of optimizations to baselines: We ported the transferable optimizations to PipeRAG [15]: memory-mapped indices, warm encoder weights in DRAM, and persistent thread and core pinning. For an isolated batch at BS=1, the optimized PipeRAG [15] reaches 0.22 s for encoding, 1.55 s for retrieval, and 0.10 s for augmentation, close to MaestroRAG. Under the steady-state bursty Azure trace of Section §5.6, it reaches 1.38 QPS against our 1.60 QPS, still runs out of memory at BS=16, and suffers head-of-line blocking because its synchronous stages cannot admit the next batch independently. The remaining benefit comes from asynchronous multi-width orchestration, worker scaling, and adaptive batching.

Impact of cold start: Finally, we evaluate cold-start overhead which consists of latency incurred in model loading, process creation by scheduler for different stages, index loading and other library loads. MaestroRAG incurs a one-time cold-start cost of 7s comprising 0.36s for encoder loading, 3.04s for vector index construction, and 3.36s for loading the generation model onto the GPU. In contrast, FlashRAG and EdgeRAG re-incurs this overhead at the beginning of every batch, highlighting a key advantage of our initialization strategy in batch inference settings.

6. Related Work

RAG Pipeline Optimization.. Prior systems such as FlashRAG [18], PipeRAG [15], TurboRAG [23], and RAGCache [17] provide modular frameworks and system-level optimizations for RAG, but primarily target datacenter deployments with abundant GPU resources. EdgeRAG [34] addresses edge scenarios but relies on on-demand embedding without systematic CPU–GPU orchestration. Hardware accelerators for RAG [5, 14, 21, 30] achieve strong speedups but complicate integration with resource-limited devices.

Edge ML Systems. Research on edge LLM deployment focuses on layer offloading [2, 35] and quantization [10, 40] to reduce memory footprint. These methods accelerate the generative model but do not address the full RAG pipeline's encoding and retrieval stages.

Heterogeneous Resource Management. Work on CPU resource management [11, 27] explores compute–memory overlap but is not tailored to RAG's unique stage characteristics. MaestroRAG bridges these gaps with characterization-driven, fine-grained orchestration designed specifically for edge RAG deployment.

7. Conclusions

RAG models, with their compact size and personalized responses, are well-suited for edge deployment. To address system-level challenges in this context, we dissect RAG pipeline to quantify how various factors impact performance on edge devices. Through detailed stage-wise characterization, we identify inefficiencies in existing solutions and leverage these insights to design a multi-width, resource-aware pipeline that better utilizes available compute. Our implementation, MaestroRAG, targets two common edge use cases: latency-critical and throughput-critical tasks. Without modifying hardware or model architecture, our hardware-aware, software-only scheme achieves up to 12× latency reduction and 5.6× throughput improvement over state-of-the-art baselines. This makes RAG more viable for edge deployment and sets a new benchmark for future research.

8. AI Usage Disclosure

Generative AI tools were used for grammar check and sentence formatting during manuscript preparation. Agentic coding tools were used for graph preparation and experimental automation. All technical content, experimental results, algorithms, and architectural designs are the original work of the authors.

References

[1] M. AI, "Llama 3.1 8b instruct model," 2024, accessed: 2024-11-23. [Online]. Available: https://catalog.ngc.nvidia.com/orgs/nvidia/teams/nemo/models/llama-3_1-8b-instruct-nemo

[2] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale," in *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*, ser. SC '22. IEEE Press, 2022.

[3] S. A. Bergman, Z. Ji, A.-M. Kermarrec, D. Petrescu, R. Pires, M. Randl, and M. de Vos, "Leveraging approximate caching for faster retrieval-augmented generation," in *Proceedings of the 5th Workshop on Machine Learning and Systems*, ser. EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 66–73. [Online]. Available: https://doi.org/10.1145/3721146.3721941

[4] B. J. Chan, C.-T. Chen, J.-H. Cheng, and H.-H. Huang, "Don't do rag: When cache-augmented generation is all you need for knowledge tasks," in *Companion Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 893–897. [Online]. Available: https://doi.org/10.1145/3701716.3715490

[5] K. Chen, R. Nadig, M. Frouzakis, N. M. Ghiasi, Y. Liang, H. Mao, J. Park, M. Sadrosadati, and O. Mutlu, "Reis: A high-performance and energy-efficient retrieval system with in-storage processing," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1171–1192. [Online]. Available: https://doi.org/10.1145/3695053.3731116

[6] S. Chen, S. Huang, S. Pandey, B. Li, G. R. Gao, L. Zheng, C. Ding, and H. Liu, "E.t.: Re-thinking self-attention for transformer models on gpus," in *SC21: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2021, pp. 1–14.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423/

[8] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini, and H. Jégou, "The faiss library," 2024.

[9] D. Franklin, "Nanollm," "https://github.com/dusty-nv/NanoLLM", 2025.

[10] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.

[11] Z. Gong, H. Ji, Y. Yao, C. W. Fletcher, C. J. Hughes, and J. Torrellas, "Graphite: optimizing graph neural networks on cpus through cooperative software-hardware techniques," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 916–931.

[12] P. Indyk and R. Motwani, "Approximate nearest neighbors: towards removing the curse of dimensionality," in *Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing*, ser. STOC '98. New York, NY, USA: Association for Computing Machinery, 1998, p. 604–613. [Online]. Available: https://doi.org/10.1145/276698.276876

[13] Intel, "Intel i9-14900k," 2025, accessed: 2024-11-23. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/236773/intel-core-i9-processor-14900k-36m-cache-up-to-6-00-ghz/specifications.html

[14] W. Jiang, M. Zeller, R. Waleffe, T. Hoefler, and G. Alonso, "Chameleon: A heterogeneous and disaggregated accelerator system for retrieval-augmented language models," *Proc. VLDB Endow.*, vol. 18, no. 1, p. 42–52, Sep. 2024. [Online]. Available: https://doi.org/10.14778/3696435.3696439

[15] W. Jiang, S. Zhang, B. Han, J. Wang, B. Wang, and T. Kraska, "Piperag: Fast retrieval-augmented generation via adaptive pipeline parallelism," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1*, ser. KDD '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 589–600. [Online]. Available: https://doi.org/10.1145/3690624.3709194

[16] Z. Jiang, H. Lin, Y. Zhong, Q. Huang, Y. Chen, Z. Zhang, Y. Peng, X. Li, C. Xie, S. Nong, Y. Jia, S. He, H. Chen, Z. Bai, Q. Hou, S. Yan, D. Zhou, Y. Sheng, Z. Jiang, H. Xu, H. Wei, Z. Zhang, P. Nie, L. Zou, S. Zhao, L. Xiang, Z. Liu, Z. Li, X. Jia, J. Ye, X. Jin, and X. Liu, "Megascale: scaling large language model training to more than 10,000 gpus," in *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*, ser. NSDI'24. USA: USENIX Association, 2024.

[17] C. Jin, Z. Zhang, X. Jiang, F. Liu, S. Liu, X. Liu, and X. Jin, "Ragcache: Efficient knowledge caching for retrieval-augmented generation," *ACM Trans. Comput. Syst.*, vol. 44, no. 1, Nov. 2025. [Online]. Available: https://doi.org/10.1145/3768628

[18] J. Jin, Y. Zhu, Z. Dou, G. Dong, X. Yang, C. Zhang, T. Zhao, Z. Yang, and J.-R. Wen, "Flashrag: A modular toolkit for efficient retrieval-augmented generation research," in *Companion Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 737–740. [Online]. Available: https://doi.org/10.1145/3701716.3715313

[19] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 453–466, 2019. [Online]. Available: https://ai.google.com/research/NaturalQuestions/

[20] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[21] C. Liu, H. Liu, D. Chen, Y. Huang, Y. Zhang, W. Xiao, X. Liao, and H. Jin, "Heterrag: Heterogeneous processing-in-memory acceleration for retrieval-augmented generation," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 884–898. [Online]. Available: https://doi.org/10.1145/3695053.3731089

[22] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[23] S. Lu, H. Wang, Y. Rong, Z. Chen, and Y. Tang, "Turborag: Accelerating retrieval-augmented generation with precomputed kv caches for chunked text," 2024. [Online]. Available: https://arxiv.org/abs/2410.07590

[24] Meta-AI, "Llama 3.2 1b model," 2024. [Online]. Available: https://huggingface.co/meta-llama/Llama-3.2-1B

[25] Microsoft Azure, "Azure public dataset," https://github.com/Azure/AzurePublicDataset, 2025, gitHub repository providing public datasets and resources from Microsoft Azure. Accessed 2025-07-31.

[26] B. Na, J. Jang, S. Park, S. Kim, J. Kim, M. S. Jeong, K. C. Kim, S. Heo, Y. Kim, and S. Yoon, "Scalable smartphone cluster for deep learning," *arXiv preprint arXiv:2110.12172*, 2021.

[27] K. Nair, A.-C. Pandey, S. Karabannavar, M. Arunachalam, J. Kalamatianos, V. Agrawal, S. Gupta, A. Sirasao, E. Delaye, S. Reinhardt *et al.*, "Parallelization strategies for dlrm embedding bag operator on amd cpus," *IEEE Micro*, 2024.

[28] NVIDIA, "Nvidia jetson agx orin series," "https://www.nvidia.com/content/dam/en-zz/Solutions/gtcf21/jetson-orin/nvidia-jetson-agx-orin-technical-brief.pdf", 2022.

[29] Nvidia, "Rtx 4090," 2025, accessed: 2024-11-23. [Online]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/

[30] R. Qin, Z. Yan, D. Zeng, Z. Jia, D. Liu, J. Liu, A. Abbasi, Z. Zheng, N. Cao, K. Ni, J. Xiong, and Y. Shi, "Robust implementation of retrieval-augmented generation on edge-based computing-in-memory architectures," in *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, ser. ICCAD '24. New York, NY, USA: Association for Computing Machinery, 2025. [Online]. Available: https://doi.org/10.1145/3676536.3676674

[31] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 1, Jan. 2020.

[32] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," *arXiv preprint arXiv:1910.01108*, 2019.

[33] L. G. Schroeder, A. Desai, A. Cuadron, K. Chu, S. Liu, M. Zhao, S. Krusche, A. Kemper, M. Zaharia, and J. E. Gonzalez, "vcache: Verified semantic prompt caching," 2025. [Online]. Available: https://arxiv.org/abs/2502.03771

[34] K. Seemakhupt, S. Liu, and S. Khan, "Edgerag: Online-indexed rag for edge devices," 2024. [Online]. Available: https://arxiv.org/abs/2412.21023

[35] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "Flexgen: high-throughput generative inference of large language models with a single gpu," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.

[36] J. Stojkovic, C. Zhang, I. n. Goiri, E. Choukse, H. Qiu, R. Fonseca, J. Torrellas, and R. Bianchini, "Tapas: Thermal- and power-aware scheduling for llm inference in cloud platforms," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1266–1281. [Online]. Available: https://doi.org/10.1145/3676641.3716025

[37] The Linux Documentation Project, *turbostat — Report processor frequency and idle statistics (man 8)*, The Linux Documentation Project, 2025, online manual page, Linux Project. [Online]. Available: https://www.linux.org/docs/man8/turbostat.html

[38] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "E5-base-v2: Text embeddings by weakly-supervised contrastive pre-training," 2022. [Online]. Available: https://huggingface.co/intfloat/e5-base-v2

[39] Wikipedia contributors, "Wikipedia corpus dump," 2024. [Online]. Available: https://dumps.wikimedia.org/

[40] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: accurate and efficient post-training quantization for large language models," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023.

[41] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin, T. Mihaylov, M. Ott, S. Shleifer, K. Shuster, D. Simig, P. S. Koura, A. Sridhar, T. Wang, and L. Zettlemoyer, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022. [Online]. Available: https://arxiv.org/abs/2205.01068